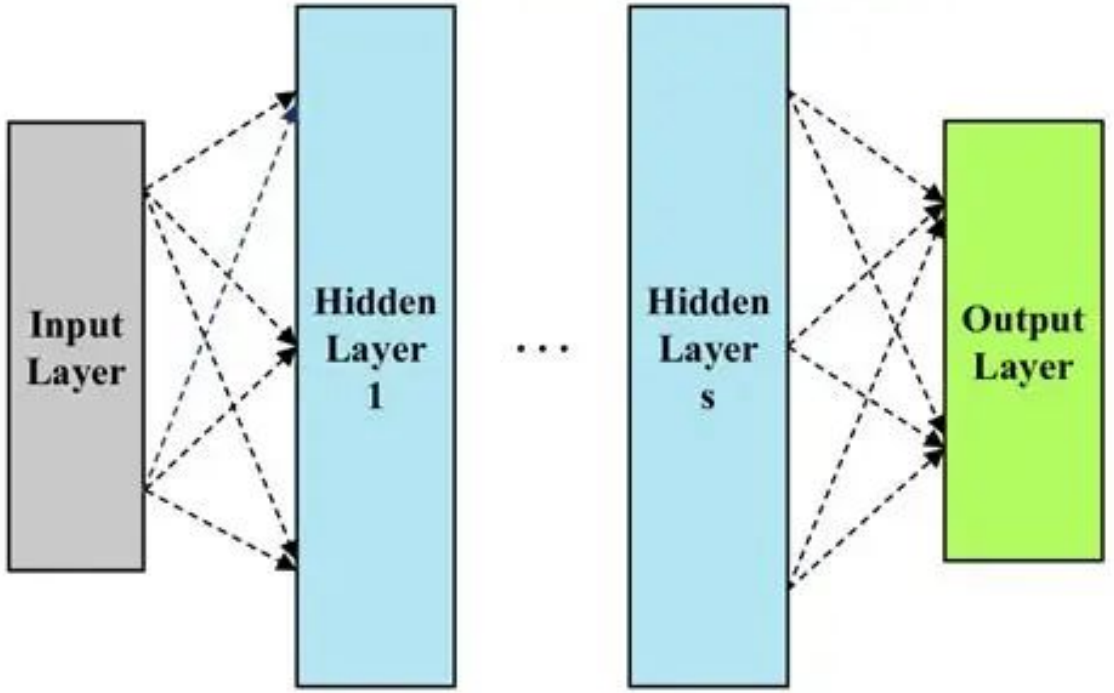
Artificial Intelligence and Machine Learning



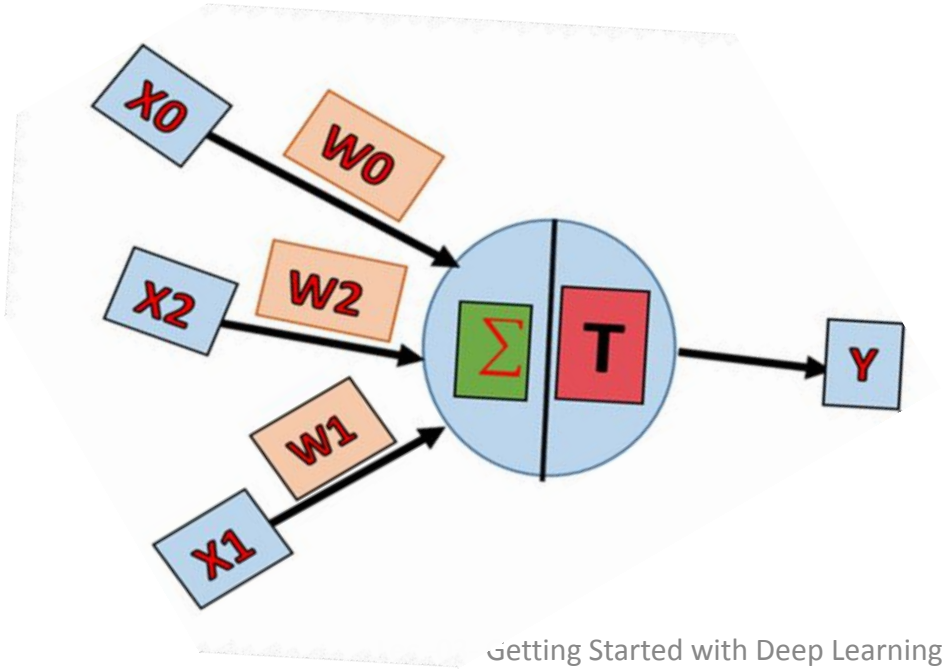
Artificial Neural Networks

Week 04- Lecture

Sunita Parajuli

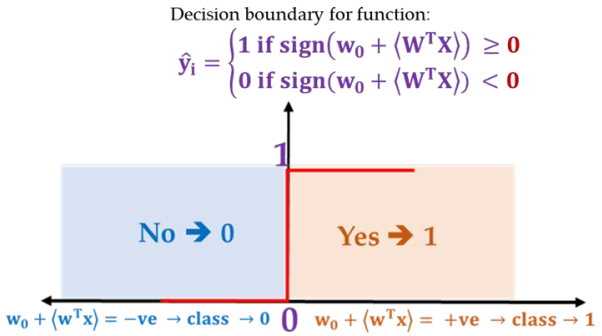
The Perceptron Model.

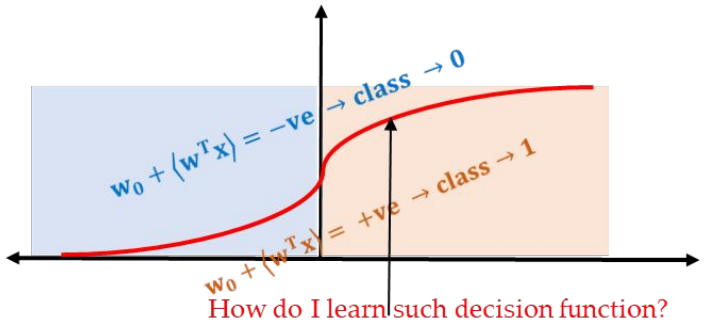
- The **Perceptron**, introduced by **Frank Rosenblatt** and later corrected by **Minsky and Papert**, extends the **McCulloch-Pitts neuron**
  - by incorporating **learnable numerical weights** and
  - an **adaptive learning process**.
  - It serves as a **Linear Binary Classifier**, dividing data into two categories based on a linear decision boundary.



Perceptron for Real Value Inputs.

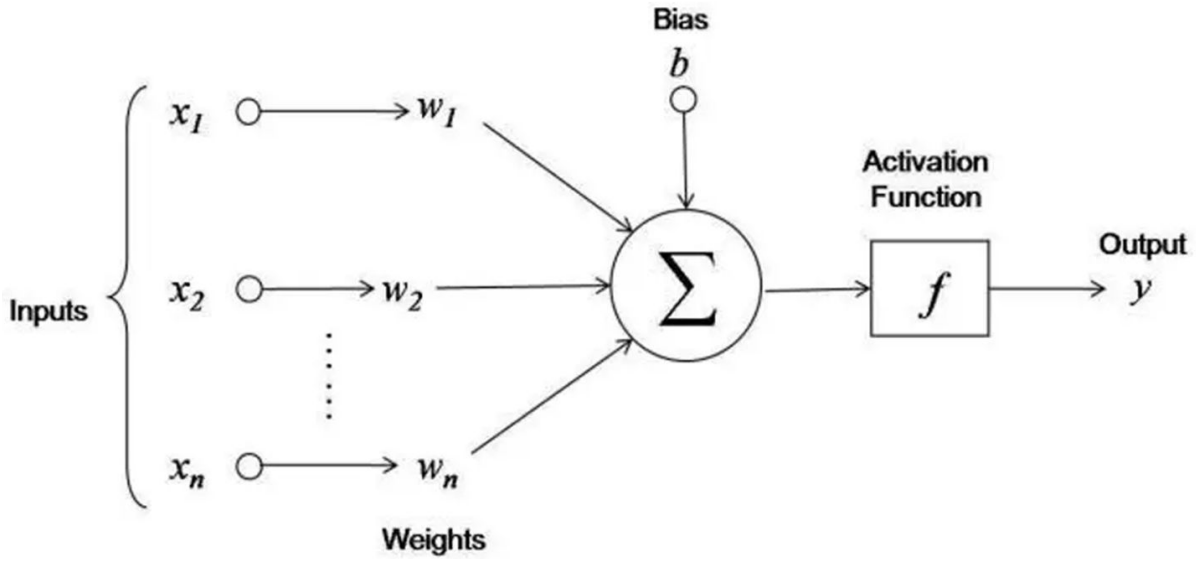
- **Step function** is unsuitable for real-valued input due to non-differentiability and binary output, which hinder learning and fine-tuning.
- The **step function** may not capture the subtleties of the data;
  - it creates a sharp boundary at 0,
    - meaning that all positive sums are classified as class 1
    - and all negative sums are classified as class 0.





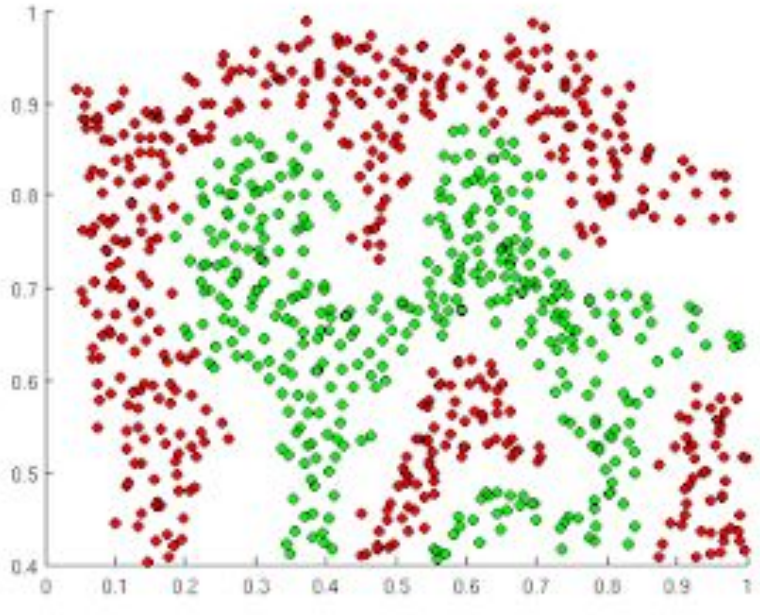
The activation function

- In General, Activation Functions are group of function that introduces **non-linearity** to the **output** of neurons.
  - The activation function does the **non-linear transformation to the input**, making it capable to learn and perform more complex tasks.



Importance of Activation Functions

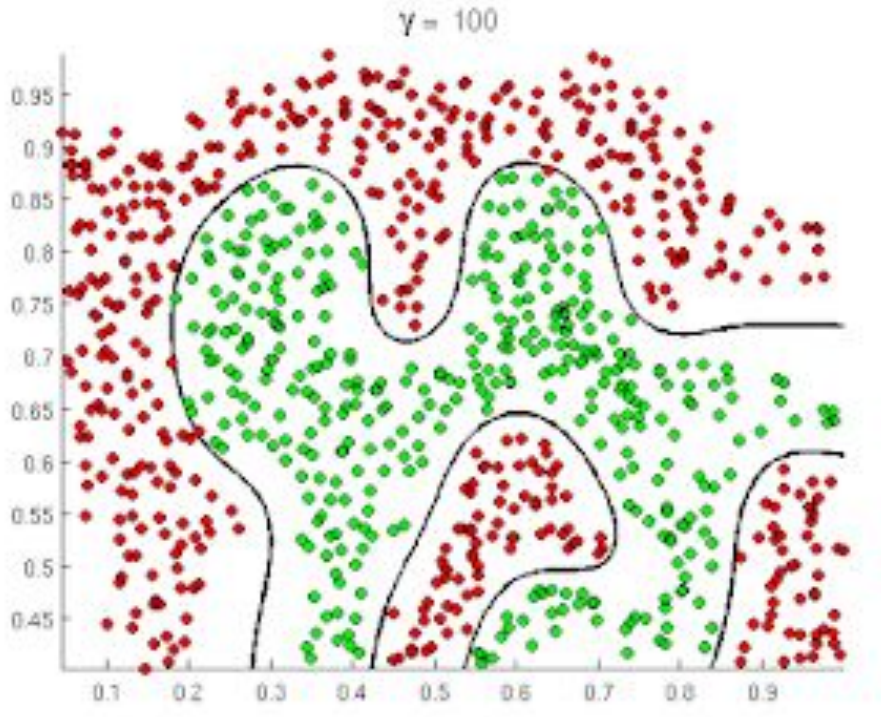
The purpose of the activation functions is to introduce non linearity to the neural network



What if we wanted to build the neural network to distinguish these two different type of data points?

Importance of Activation Functions

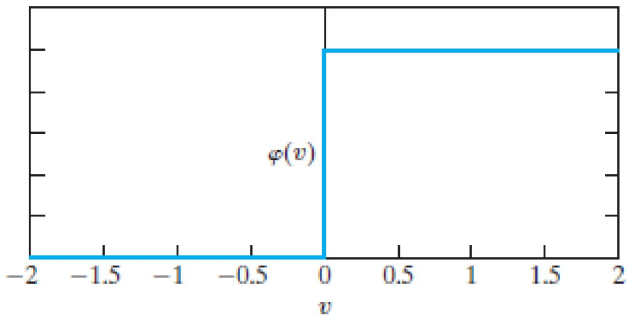
The purpose of the activation functions is to introduce non linearity to the neural network



Activation functions allow us to approximate such complex non linear functions

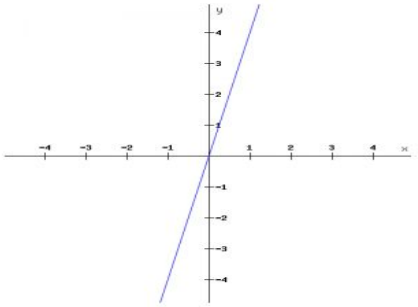
Some common Activation Functions

Threshold Function



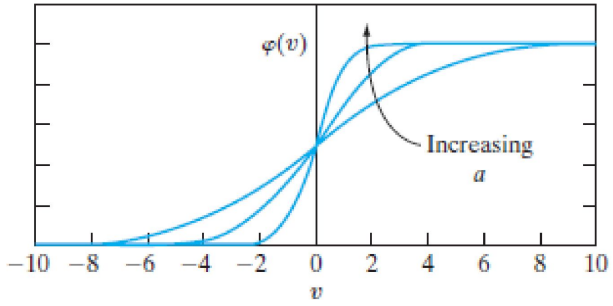
$$\varphi(x) = \begin{cases} 1 & if \quad x \geq c \\ 0 & if \quad x < c \end{cases}$$

Linear Function



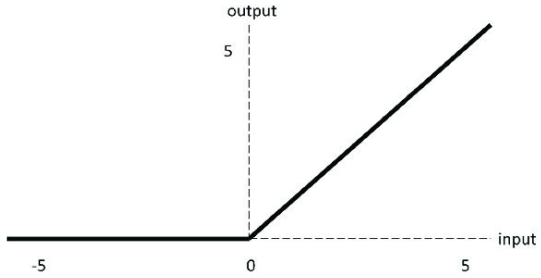
$$f(x) = ax + b$$

Sigmoid Functions



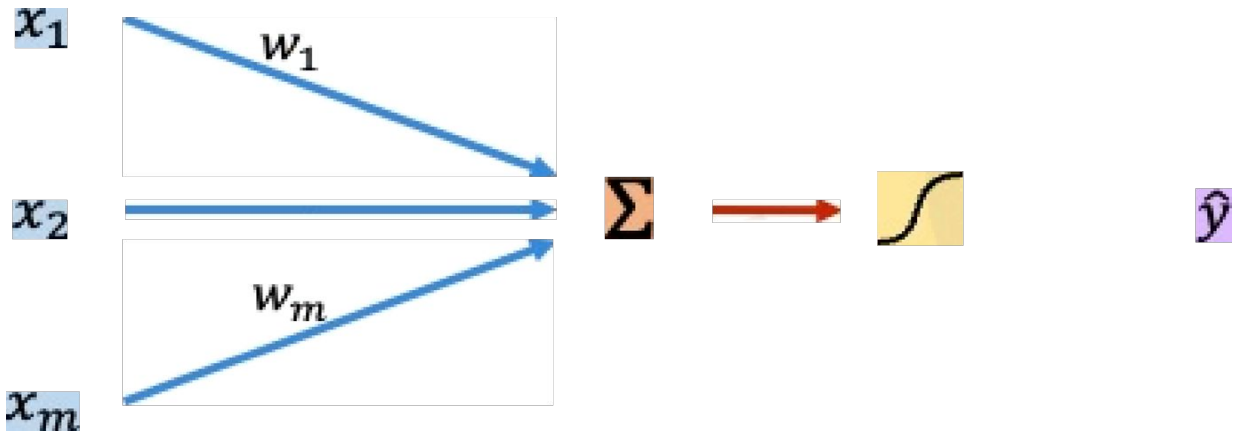
$$\varphi(x) = \frac{1}{1 + e^{-ax}}$$

Relu Funcrion



$$f(x) = \max(0, x)$$

The Perceptron :Forward Propagation



Output

Linear combination of inputs

$$\hat{y} = g\left(\sum_{i=1}^{m} x_i \, w_i\right)$$

Non-linear activation function

Building Neural Network with Perceptrons

Limitations of Perceptron

x_1	x_2	x_1 xor x_2
0	0	0
0	1	1
1	0	1
1	1	0

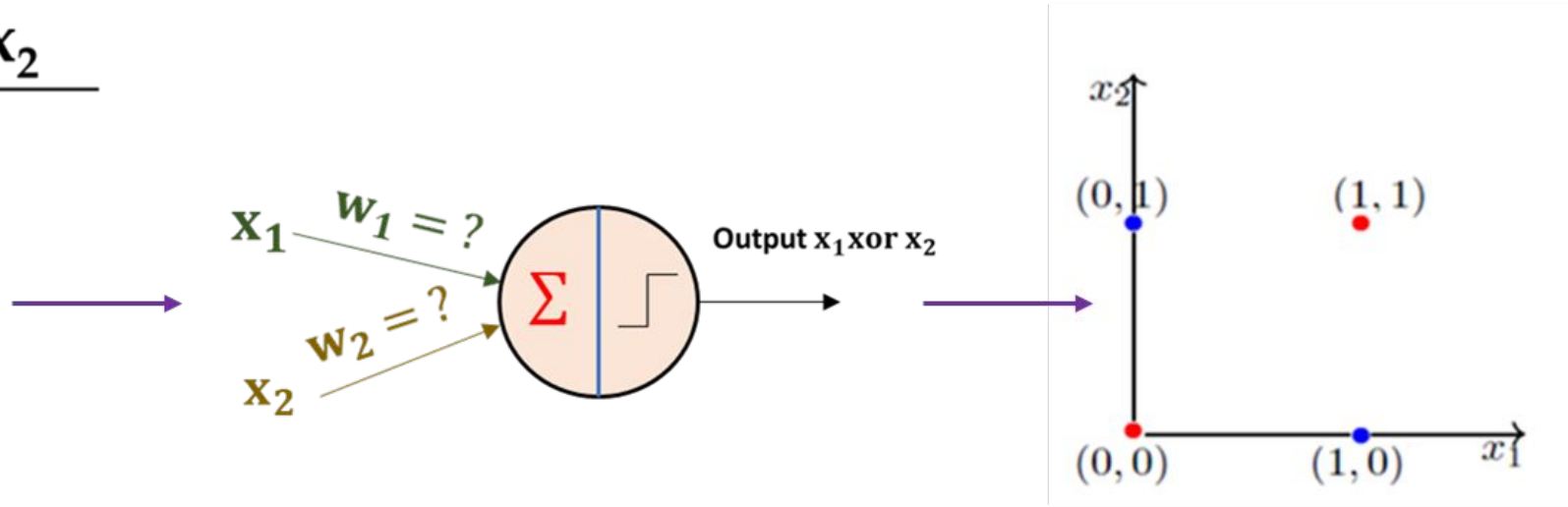


Fig: Not such value of w′s could be found, Thus No Decision Boundary Could be determined.

- Could not classify the non linearly separable data
- But we also realized that if we arrange multiple perceptrons in layers it can, it can solve such nonlinear problems

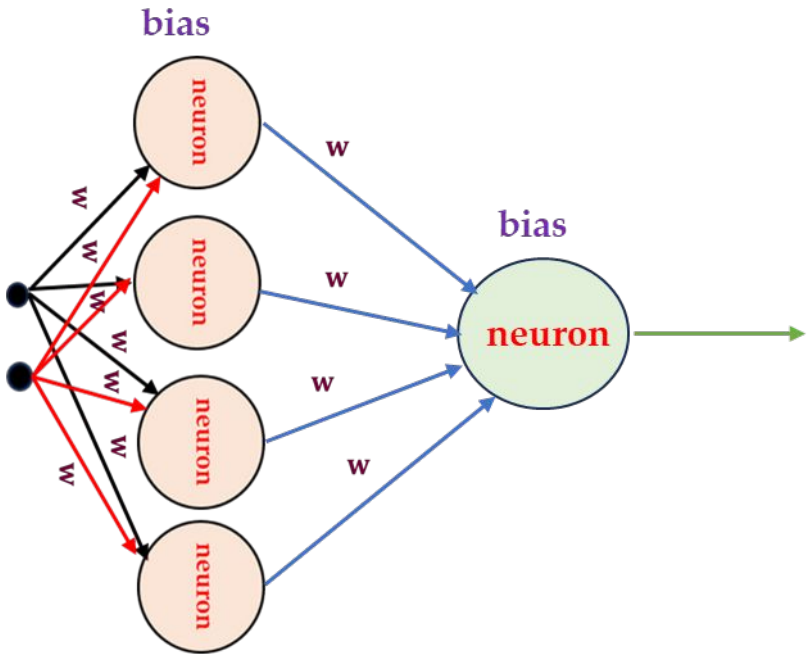
Neural Network and The Theory of Universal Approximation

- Neural networks with nonlinear activation functions can approximate any non linear/ complex function.

- How many neurons?

  2^n neurons in one hidden layer (n= inputs)

  For a Boolean function:
  2^2 = 4 neurons in one hidden layer



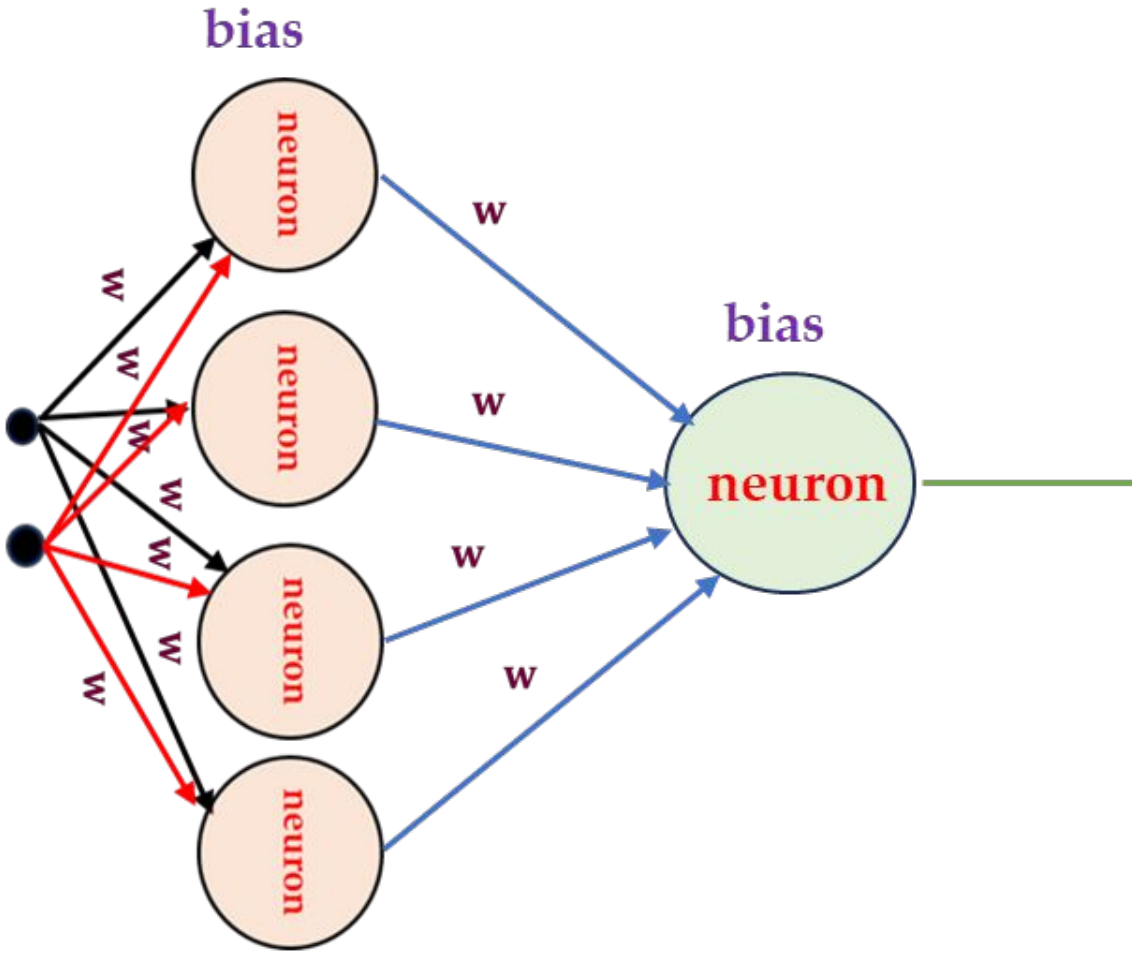
Single Layer Neural Network

For the Hidden layer:

$$z_i = w_{0i}^{(1)} + \sum_{j=1}^{m} x_j w_{ji}^{(1)}$$

For the Final Output:

$$\hat{y}_i = g\left(w_{0i}^{(2)} + \sum_{j=1}^{d_1} g(z_j) w_{ji}^{(2)}\right)$$



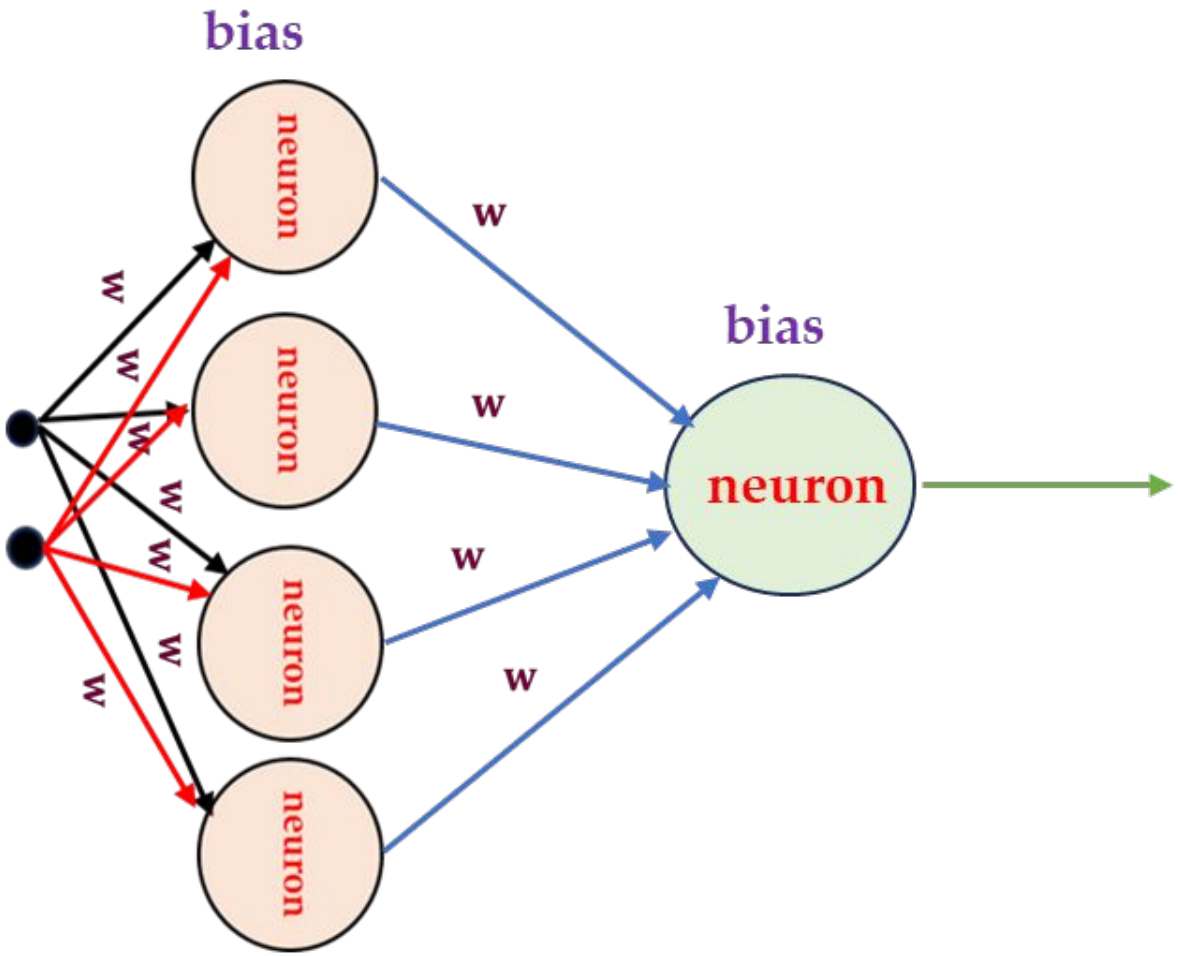
As all the nodes from the out layer are densely connected to the input it is called Dense Layer

Challenges of one hidden layer network

2^n neurons may work well for Boolean input but does not work the same for real world problems

As we may need millions of neurons in the hidden layer which is computationally inefficient!!!
(eg. Dog vs Cat classification)

Solution: Using deeper neural networks aka Multi Layered Neural Network!!



Deep Neural Networks

Why go deeper?

- Inherently compositional - builds complex patterns from simple ones
- More expressive with same parameter count
- Deep networks efficiently represent hierarchical structures found in real-world data

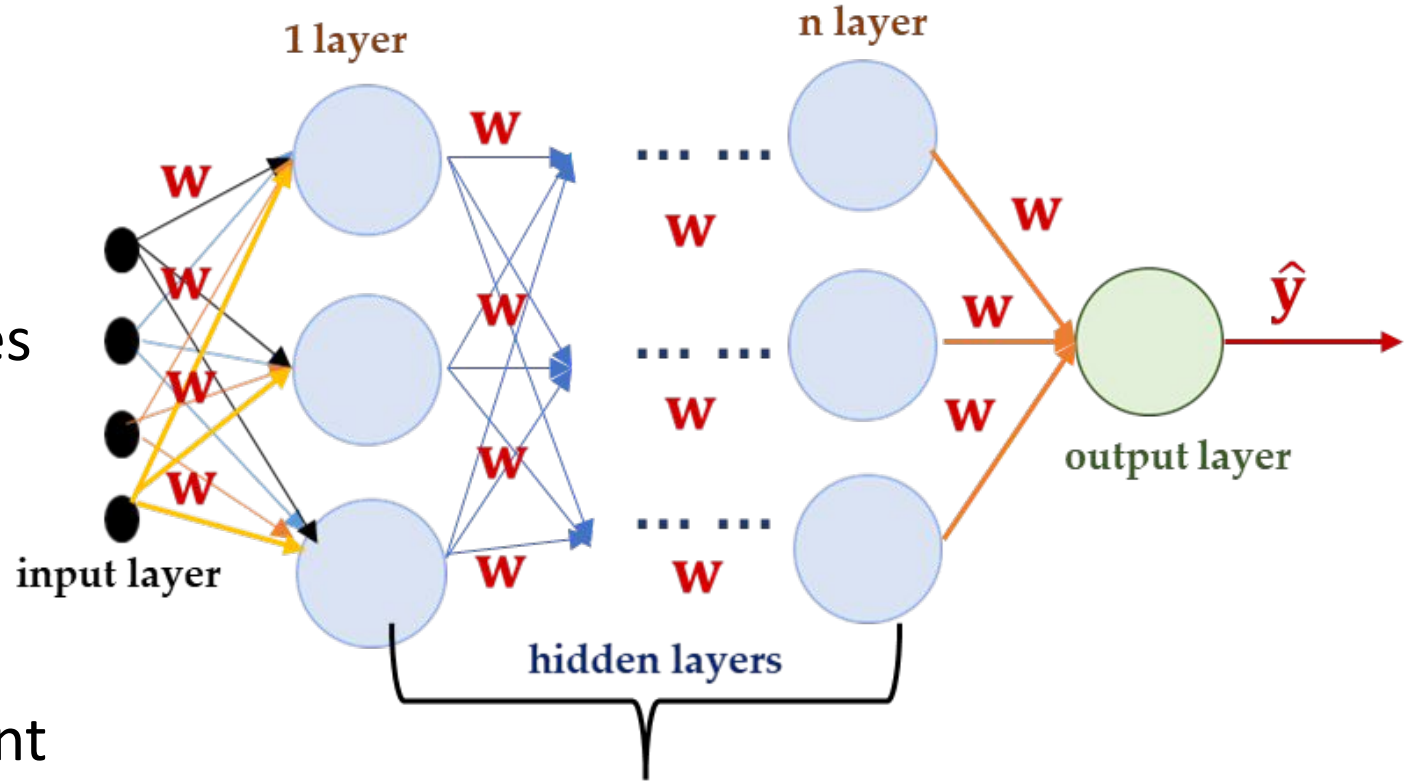
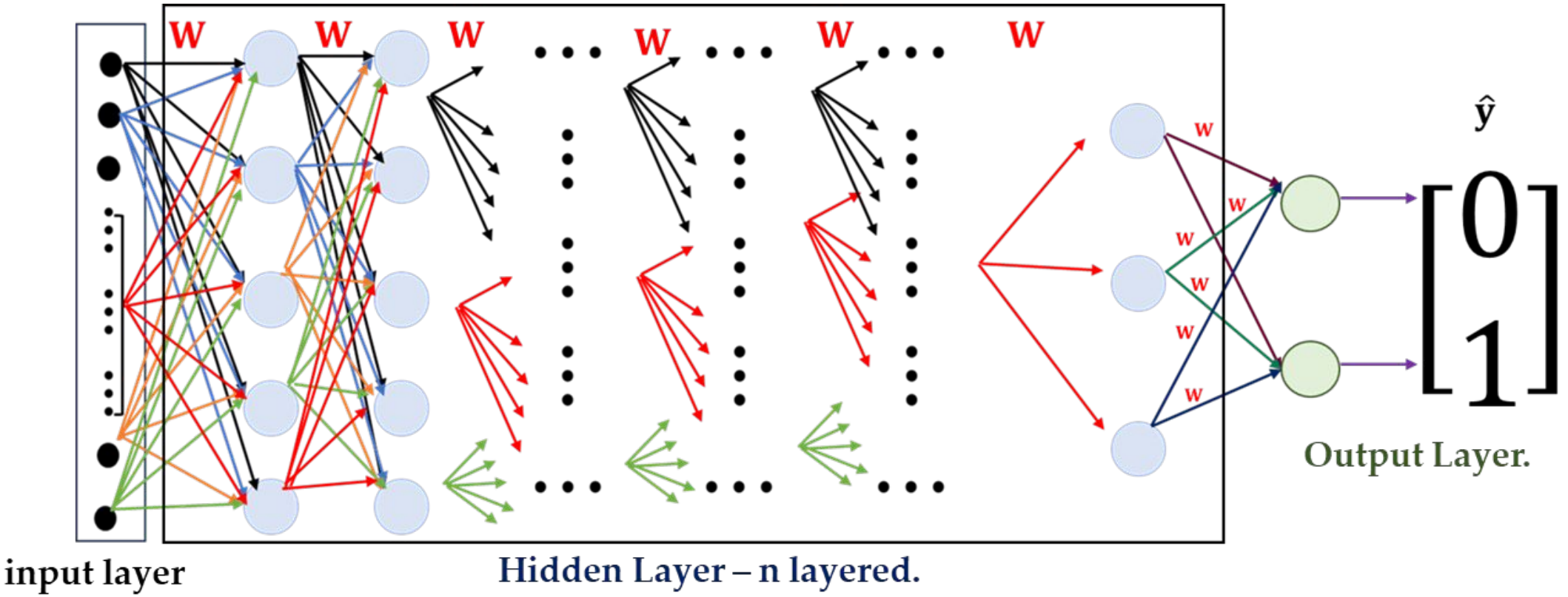
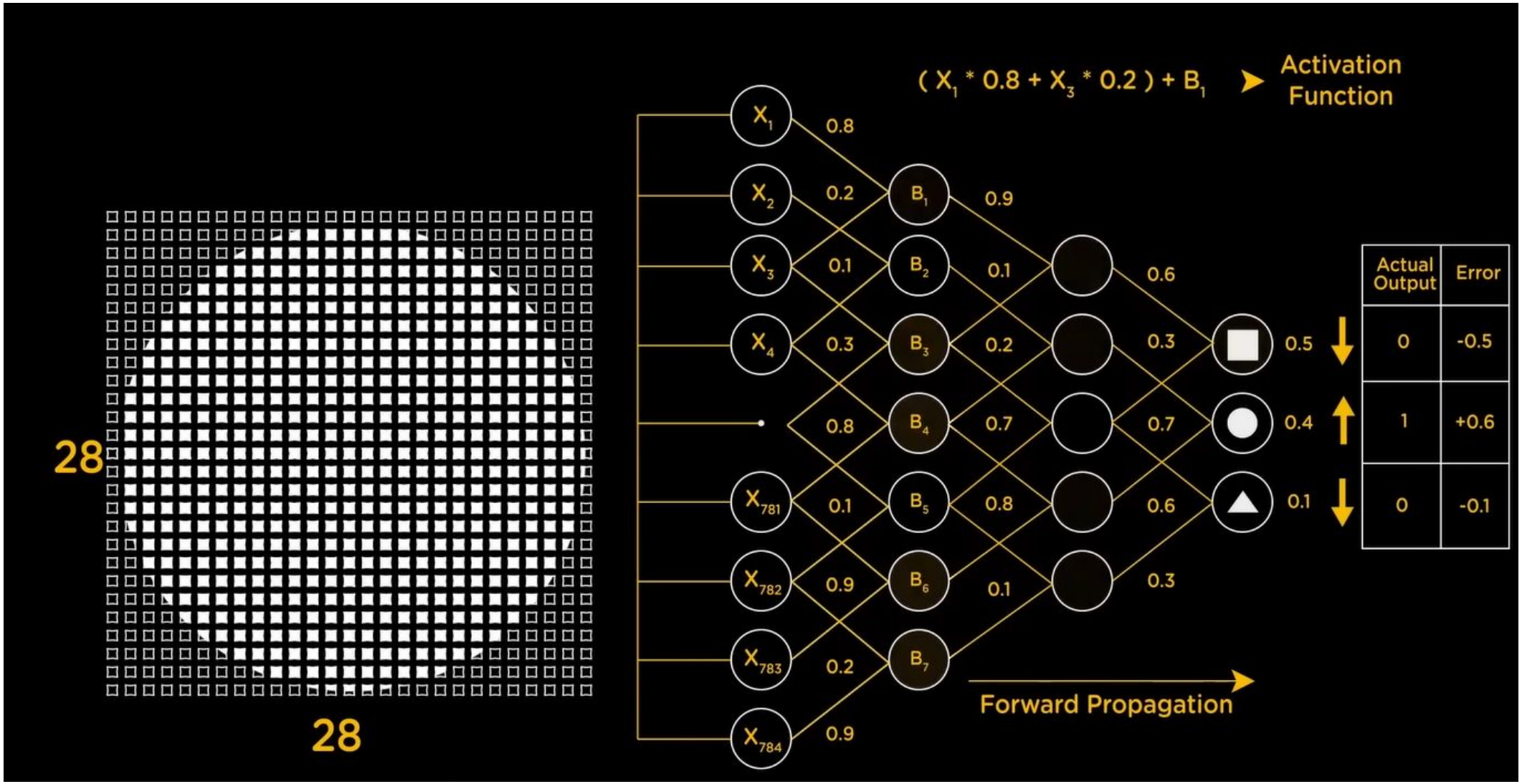


Fig: A deep network with n hidden layers.

Deep Neural Network Architecture



How DNN work! (An example)



How DNN work! (An example)

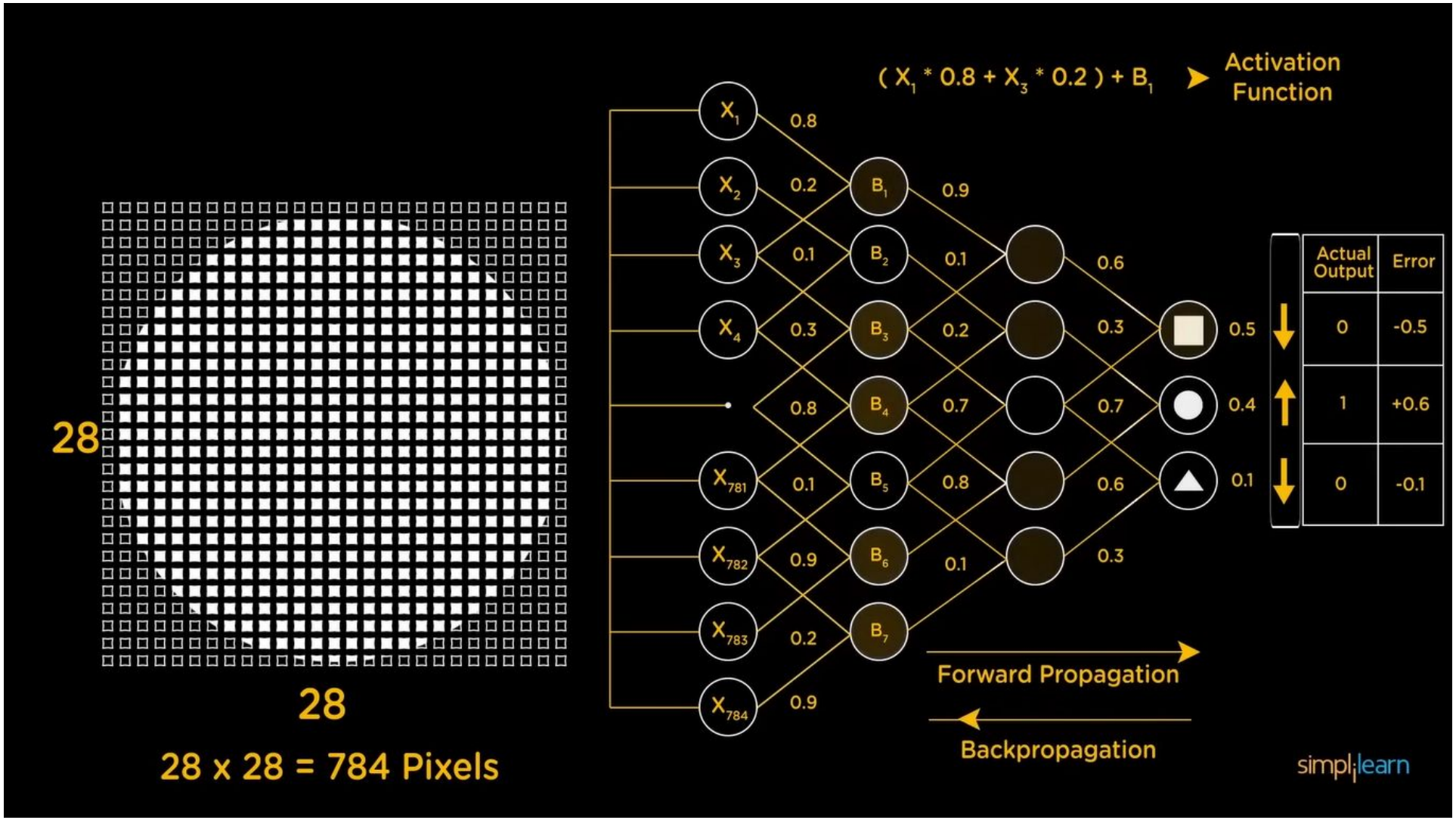


image from internet, subjected to copyrights!

Quantifying Loss (Loss Function)

- The loss function measures the discrepancy between the model's predictions and the actual values.

$$\mathcal{L}(\hat{y}, y) = \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

Predicted Actual

Empirical Loss (Cost)

Measures the average loss over the entire dataset:
Also known as :

Objective Function /
Cost Function /
Empirical Risk

$$J(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

Choosing the right Cost function

Cost Function	Formula	When to Use
Binary Cross-Entropy	$\mathcal{L}(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$	For binary classification problems where output is a probability between 0 and 1
Categorical Cross-Entropy	$\mathcal{L}(\hat{y}, y) = -\sum_{j=1}^{C} y_j \log(\hat{y}_j)$	For multi-class classification problems where outputs are probabilities across multiple classes
Mean Squared Error	$\mathcal{L}(\hat{y}, y) = \frac{1}{n} \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})^2$	For regression problems predicting continuous numerical values

Empirical Risk Minimization

- We want to find the network weights that achieve the lowest cost

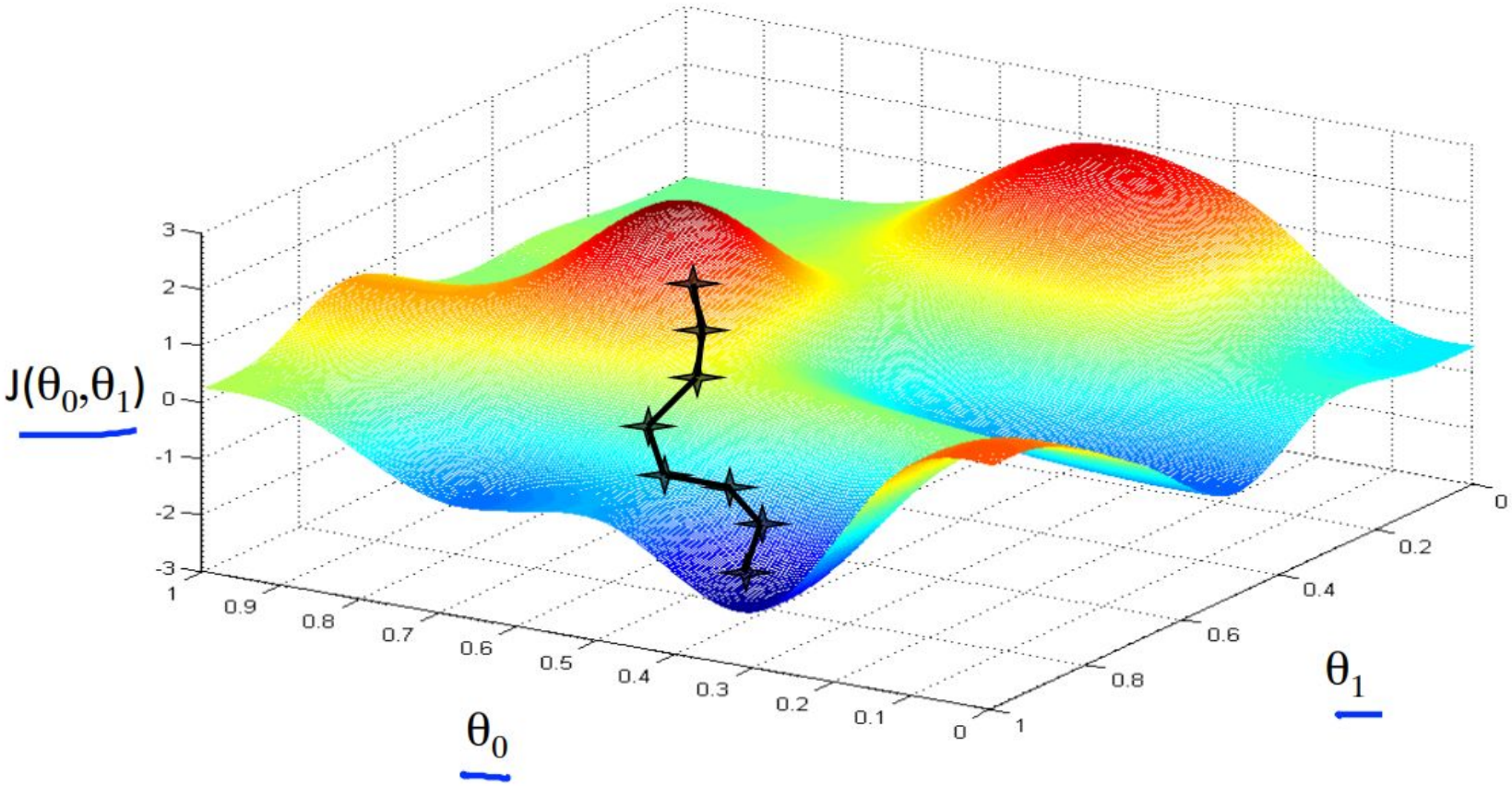
$$\mathbf{W}^* = \arg\min_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

$$\mathbf{W}^* = \arg\min_{\mathbf{W}} J(\mathbf{W})$$

Remember:

$$\boldsymbol{W} = \{\boldsymbol{W}^{(0)}, \boldsymbol{W}^{(1)}, \dots\}$$

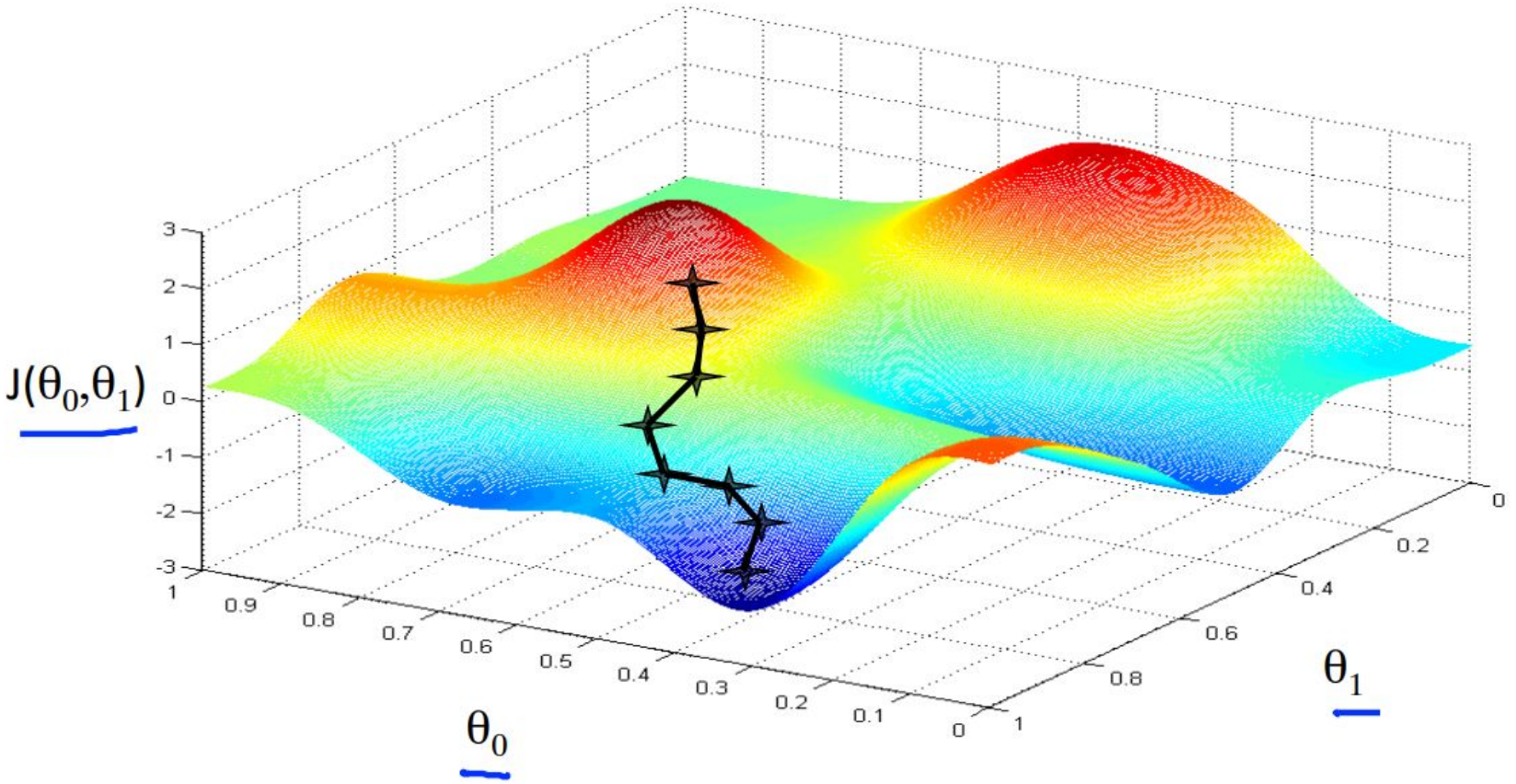
Empirical Risk Minimization



The Cost is the function of network weights w/θ

Our Goal is to minimize it

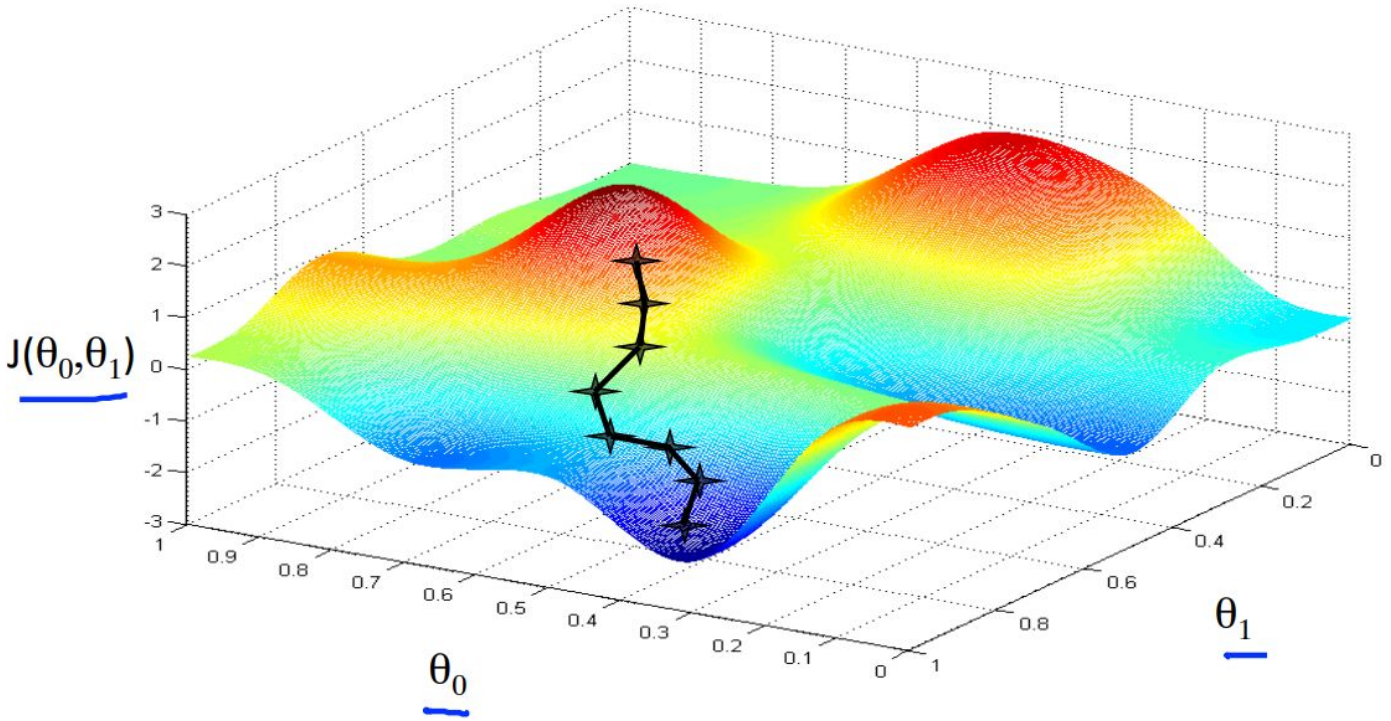
Empirical Risk Minimization



How?

Remember Gradient Descent?

Empirical Risk Minimization



Gradient Descent Algorithm

1. Initialize weights randomly
2. Loop until convergence:
3. Compute gradient, $\partial J(W)/\partial W$
4. Update weights, $W \leftarrow W - \alpha \cdot \partial J(W)/\partial W$
5. Return weights

Computing Gradient in DNN

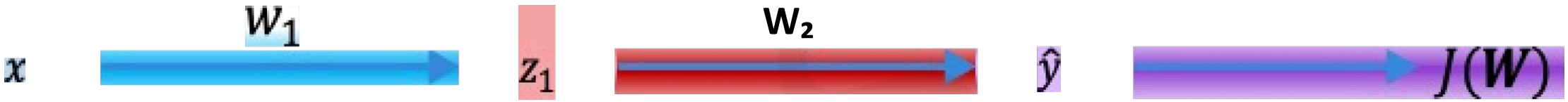
$$\frac{\partial J(\mathbf{W})}{\partial w_2} = \frac{\partial J(\mathbf{W})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2}$$

Computing Gradient in DNN

$$\frac{\partial J(\mathbf{W})}{\partial w_1} = \frac{\partial J(\mathbf{W})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_1}$$

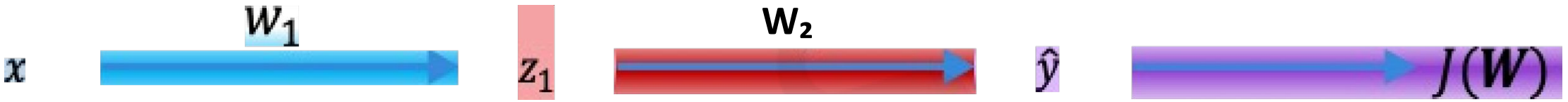
Apply Chain Rule Here

Computing Gradient in DNN



$$\frac{\partial J(\mathbf{W})}{\partial w_1} = \frac{\partial J(\mathbf{W})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

Computing Gradient in DNN



$$\frac{\partial J(\mathbf{W})}{\partial w_1} = \frac{\partial J(\mathbf{W})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

Repeat this for the weights of every layers using gradient from later layers!

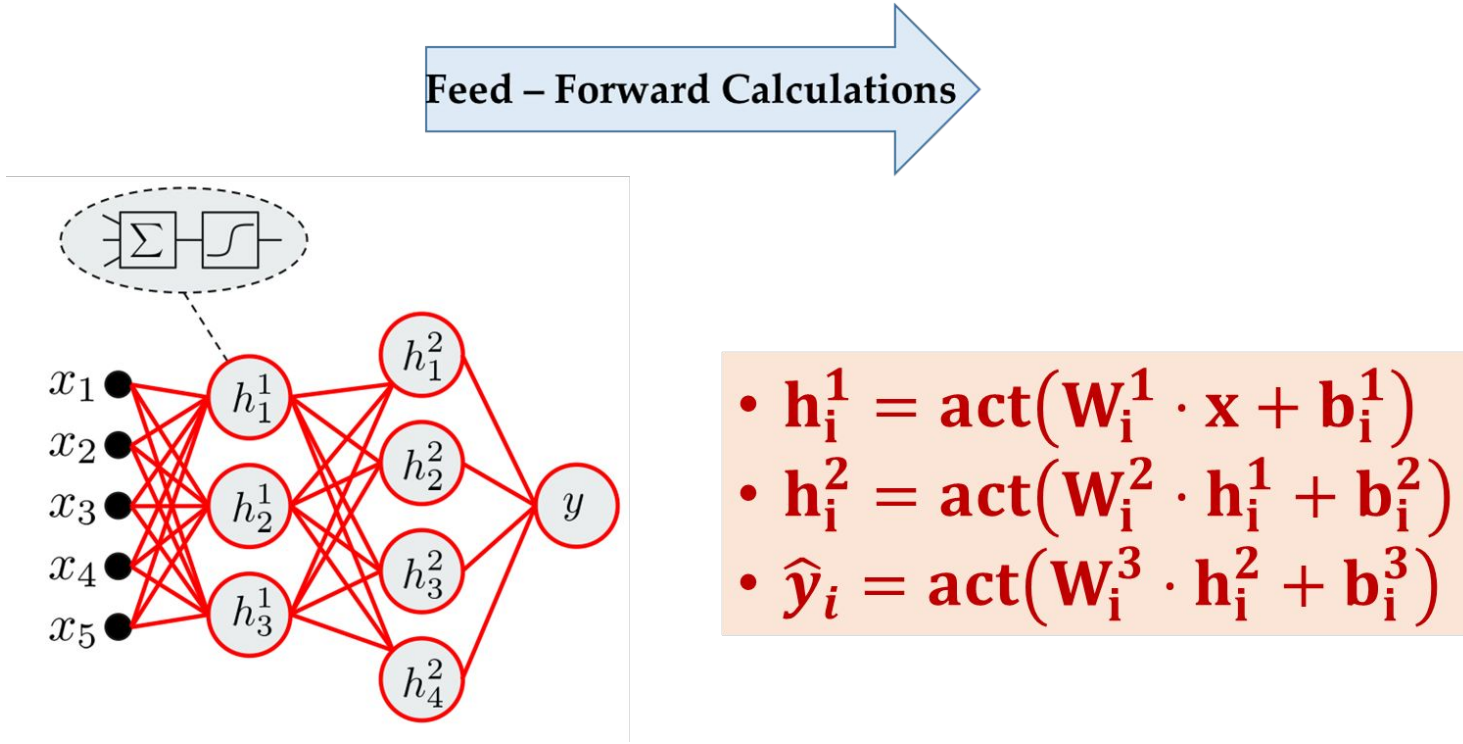
Back propagation Algorithm

The training proceeds in two phases:

- Forward Pass
- Backward Pass

Forward Pass:

- In this phase, the synaptic weights of the network are fixed and the input signal is propagated through the network, layer by layer, until it reaches the output.



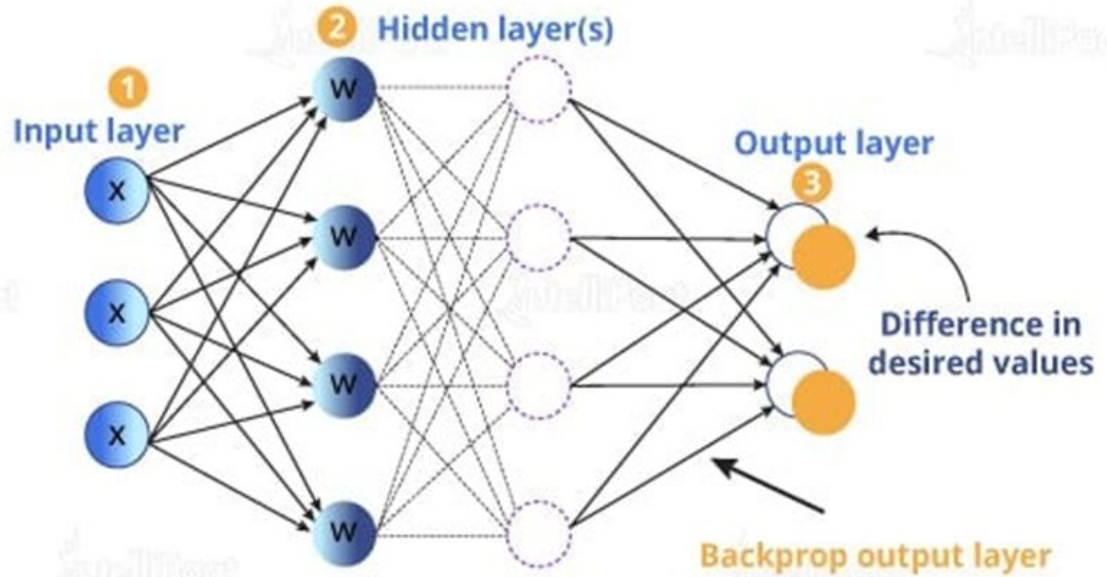
- $\mathbf{h_i^1 = act\left(W_i^1 \cdot x + b_i^1\right)}$
- $\mathbf{h_i^2 = act\left(W_i^2 \cdot h_i^1 + b_i^2\right)}$
- $\mathbf{\hat{y}_i = act\left(W_i^3 \cdot h_i^2 + b_i^3\right)}$

Backward Pass

In this phase, an error signal is produced by comparing the output of the network with a desired response.

The resulting error signal is propagated in backward direction through the network, again layer by layer.

In this second phase, successive adjustments are made to the synaptic weights of the network

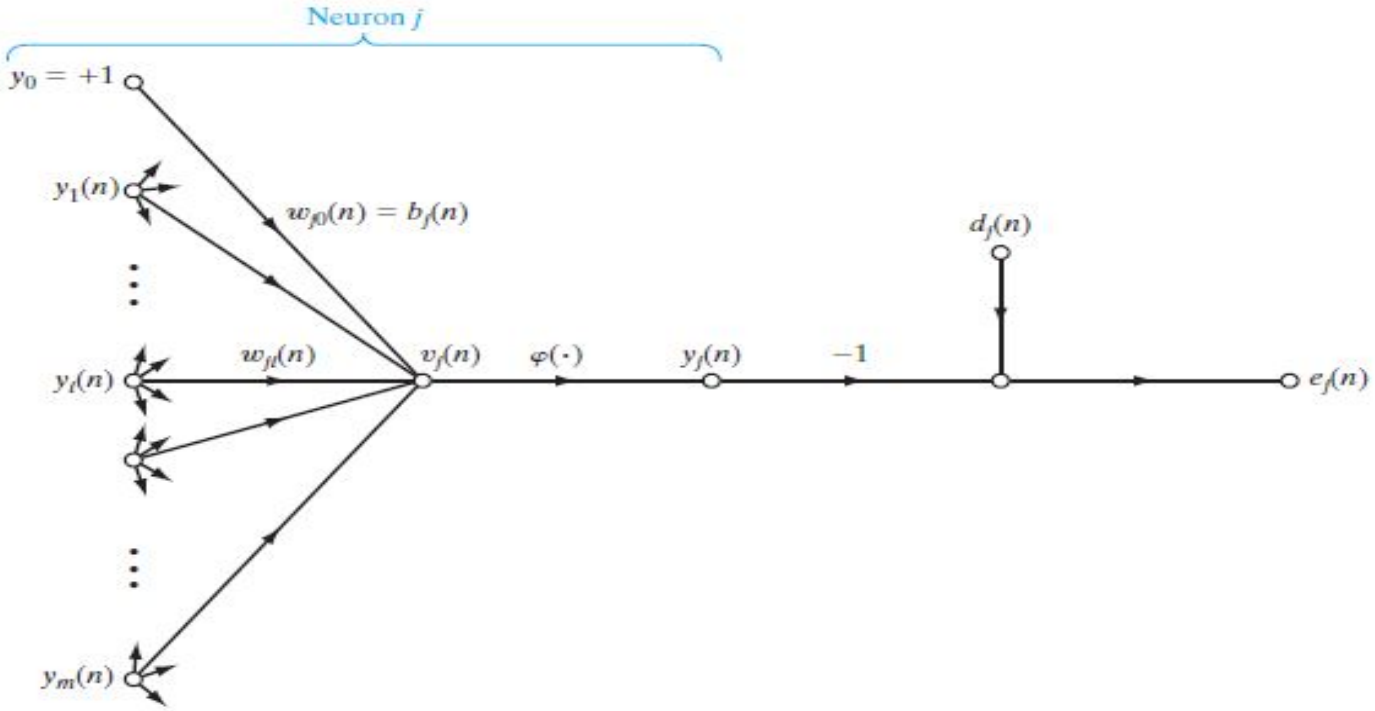


Backpropagation Algorithm

- The error signal at the output of neuron j at iteration n is given by

$$e_j(n) = d_j(n) - y_j(n)$$

where $d_j(n)$ is desired output and $y_j(n)$ is output produced by MLP

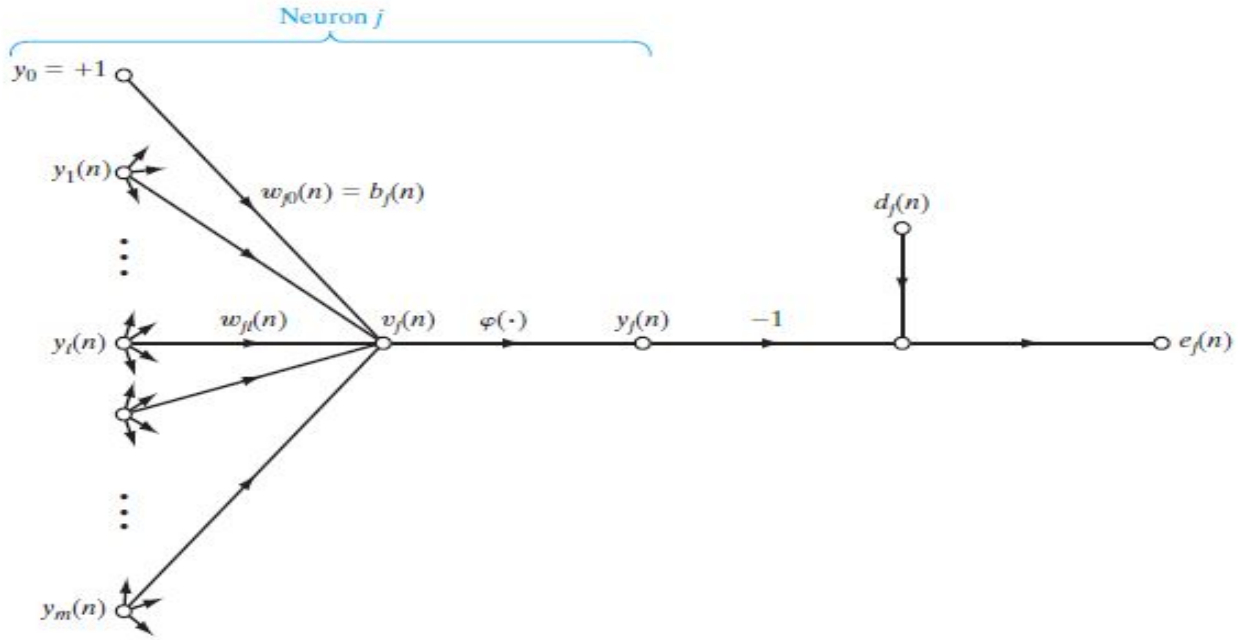


Backpropagation Algorithm

The total cost E(n) for all the neurons in the output layer is therefore

$$E(n) = \frac{1}{2}\sum_{j \in C} e_j^2(n)$$

where C is the set of neurons in output layer



Backpropagation Algorithm

- Let N be the total number of training vectors (examples). Then the average squared error is:

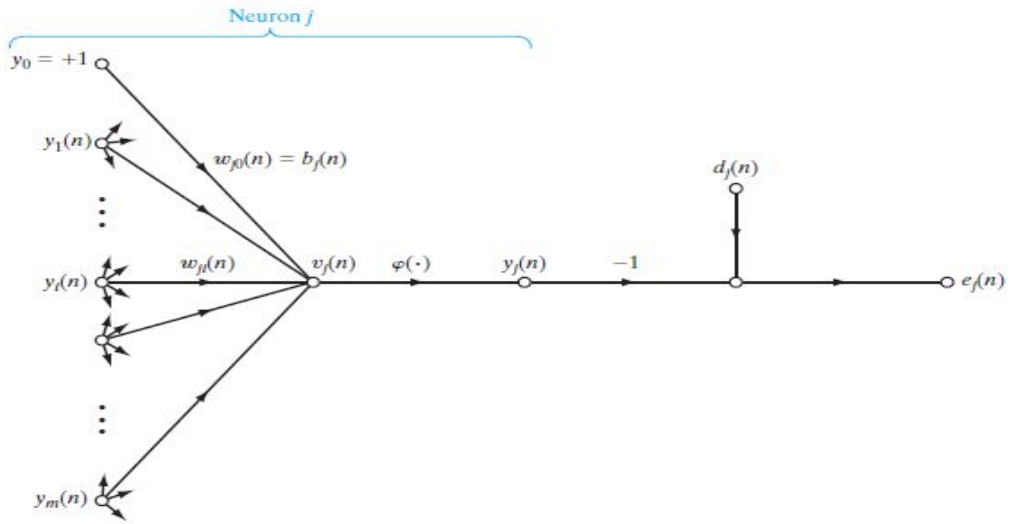
$$E_{avg} = \frac{1}{N}\sum_{n=1}^{N} E(n)$$

- Consider the neuron j ,The input signal $v_j(n)$ and output $y_j(n)$ of neuron j is given by:

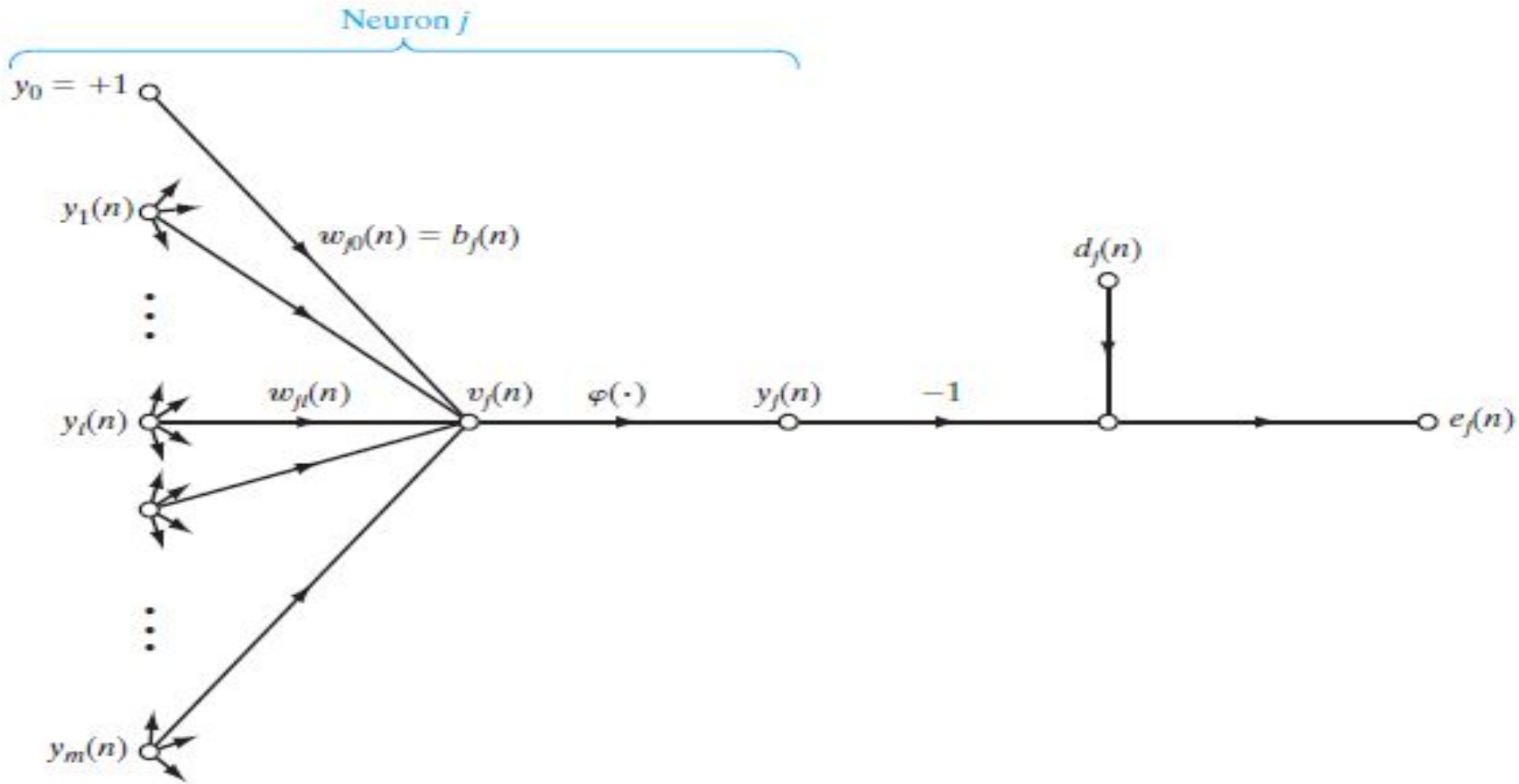
$$v_j = \sum_{i=0}^{m} w_{ji}(n) y_i(n)$$

$$y_j(n) = \varphi_j(v_j(n))$$

where y_i *is output of neuron i and* w_{ji} *is weight of link fron neuron i to j*



Back propagation Algorithm



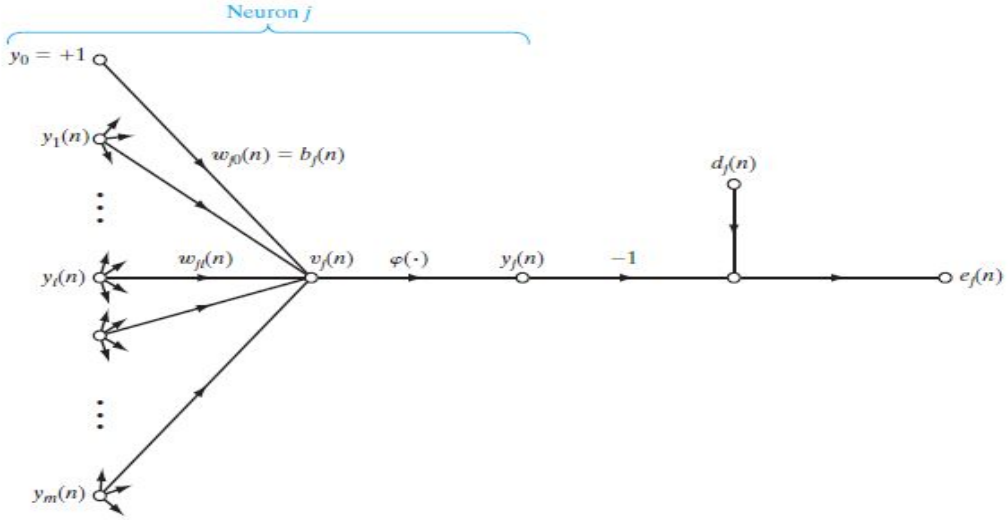
Backpropagation Algorithm

- The correction $\Delta w_{ji}(n)$ made to the weight is proportional to the partial derivative $\partial \mathrm{E(n)}/\partial w_{ji}(n)$ of instantaneous error
- Using the chain rule of calculus, this gradient can be expressed as follows:

$$\frac{\partial E(n)}{\partial w_{ji}(n)} = \frac{\partial E(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)} \qquad (1)$$

- We can get following partial derivatives

$$\frac{\partial E(n)}{\partial e_j(n)} = e_j(n) \qquad \frac{\partial e_j(n)}{\partial y_j(n)} = -1 \qquad \frac{\partial y_j(n)}{\partial v_j(n)} = \varphi_j'(v_j(n))$$



Backpropagation Algorithm

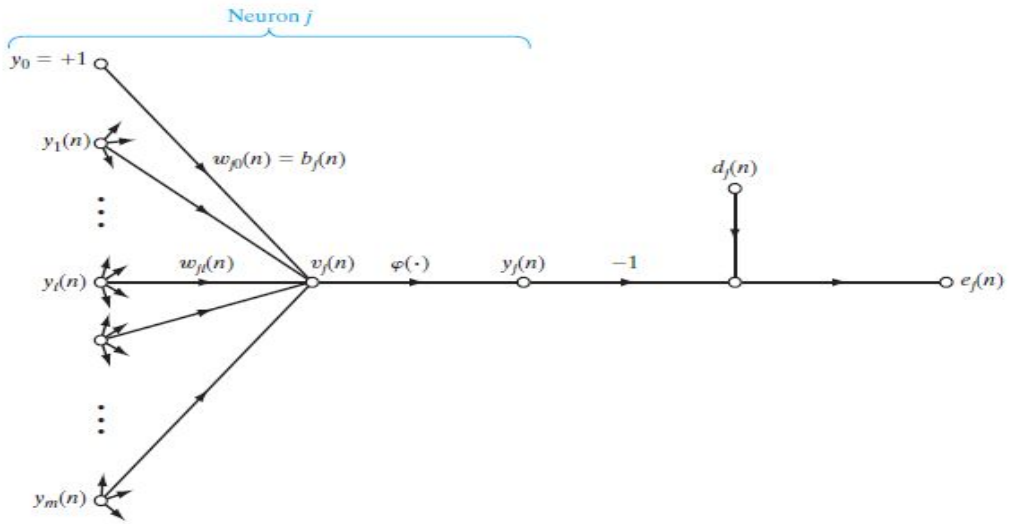
$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = y_i(n)$$

- Putting all partial derivatives in equation 1, we get

$$\frac{\partial E(n)}{\partial w_{ji}(n)} = -e_j(n)\varphi_j'(v_j(n))y_i(n) \qquad (2)$$

- The correction applied to the weight is defined by

$$\Delta w_{ji} = -\alpha \frac{\partial E(n)}{\partial w_{ji}(n)} \qquad (3)$$



Backpropagation Algorithm

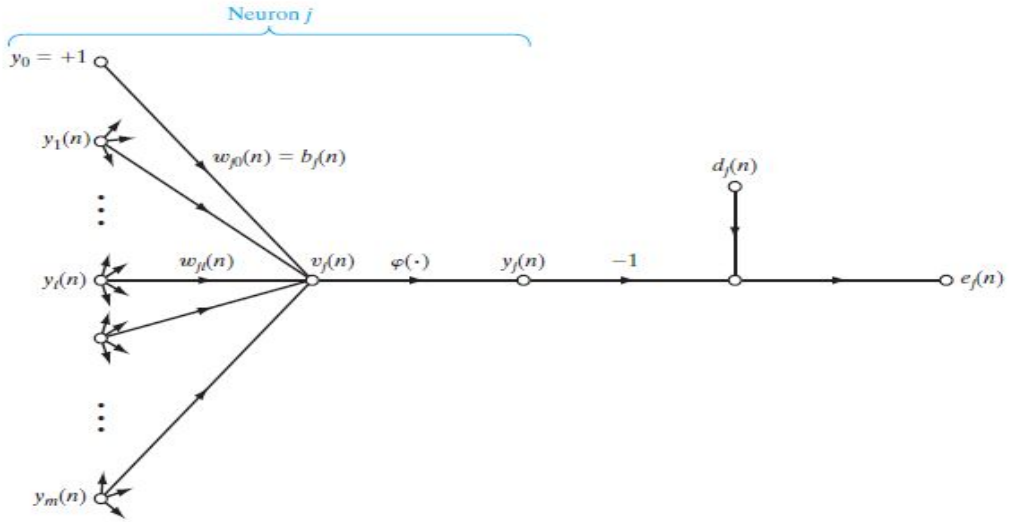
- Using equation (2), equation (3) can be written as

$$\Delta w_{ji} = \alpha \underbrace{e_j(n)\varphi_j'(v_j(n))}_{\delta_j(n)} y_i(n) \qquad (4)$$

- This can be written as

$$\Delta w_{ji} = \alpha \delta_j(n) y_i(n)$$

$$\text{where } \delta_j(n) = e_j(n)\varphi_j'(v_j(n)) \qquad (5)$$

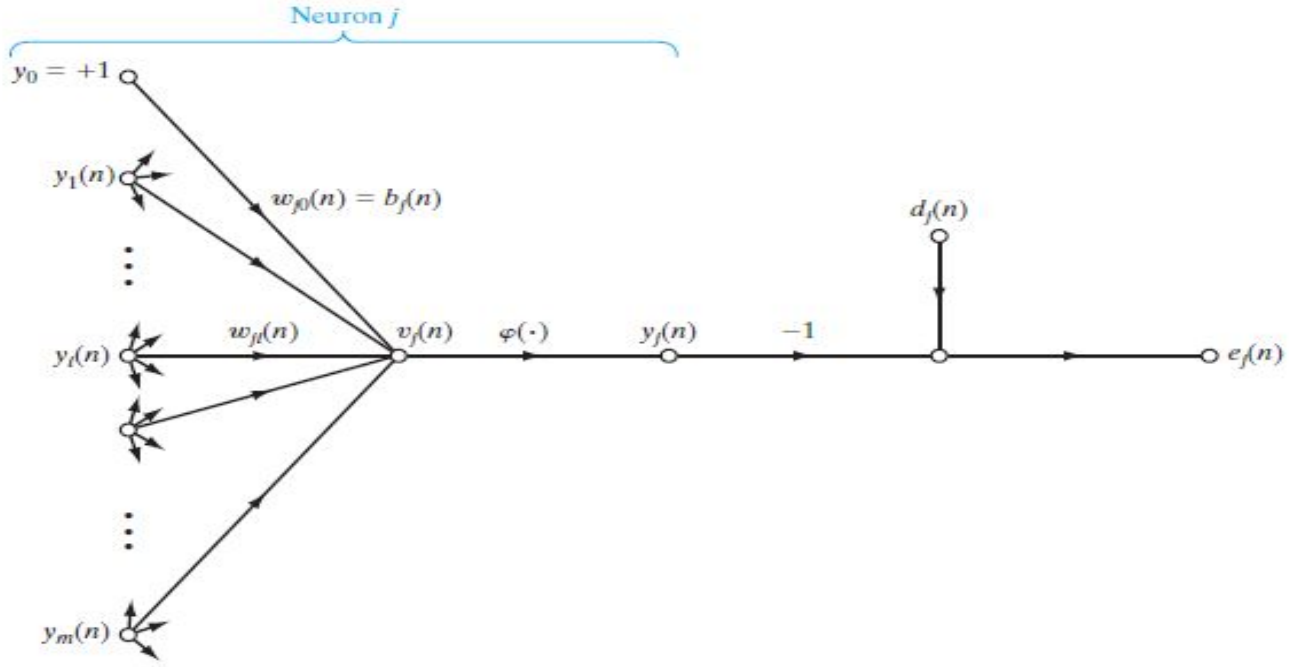


- The error term in equation 4 and 5 depends upon location of neuron in the MLP. There can be two possible scenarios.

Backpropagation Algorithm

Case I: Neuron j is output layer neuron

- The desired response $d_j(n)$ for the neuron j is directly available. Computation of the error $e_j(n)$ is straightforward in this case. We can use equation 4 or 5 to calculate weight update term.



$$\delta_j(n) = e_j(n)\,\phi_j'\big(v_j(n)\big)$$

where:

$$e_j(n) = d_j(n) - y_j(n)$$

Therefore :

$$\delta_j(n) = \big(d_j(n) - y_j(n)\big)\,\phi_j'\big(v_j(n)\big)$$

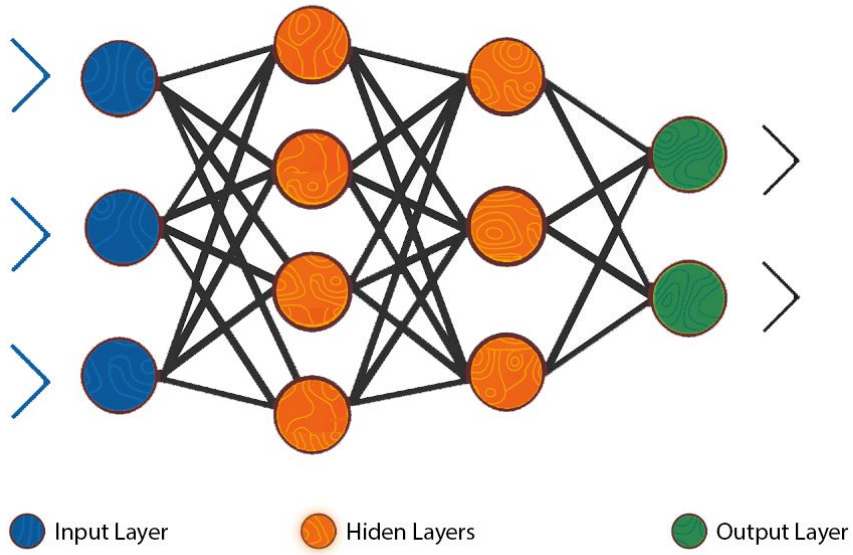
Backpropagation Algorithm

Case II: Neuron j is hidden layer neuron

- There is no desired response available for neuron j. The error signal for a hidden neuron must be determined recursively in terms of the error signals of all neurons connected to it as below:

$$\text{where } \delta_j(n) = e_j(n)\varphi_j'(v_j(n))$$

$$\delta_j(n) = \varphi_j'(v_j(n)) \sum_{k} \delta_k(n) w_{kj}(n)$$



Backpropagation Algorithm

Algorithm

1. Initialize all weights and biases in *network*
2. While terminating condition is not satisfied
3. for each training tuple **X** in *D*
4. for each input layer unit *j*: $y_j = x_j$; // output of an input unit is its actual input value
5. for each hidden or output layer unit *j*

   compute the net input of unit j with respect to the previous layer,

   $$v_j = \sum_i w_{ji} y_i$$

   compute the output of each unit j

   $$y_j = \frac{1}{1 + e^{-v_j}}$$

Learning

Algorithm

6. for each unit j in the output layer

$$\delta_j = e_j \varphi_j'(v_j) = \varphi_j'(v_j)(d_j - y_j)$$

7. for each unit j in the hidden layers, from the last to the first hidden layer

$$\delta_j = \varphi_j'(v_j) \sum_{\mathrm{k}} \delta_k w_{kj}$$

8. for each weight w_{ji} in *network*

9.

$$\Delta w_{ji} = \alpha \delta_j y_i$$

$$w_{ji} = w_{ji} + \Delta w_{ji}$$

//Weight Update

Gradient Descent in Practice

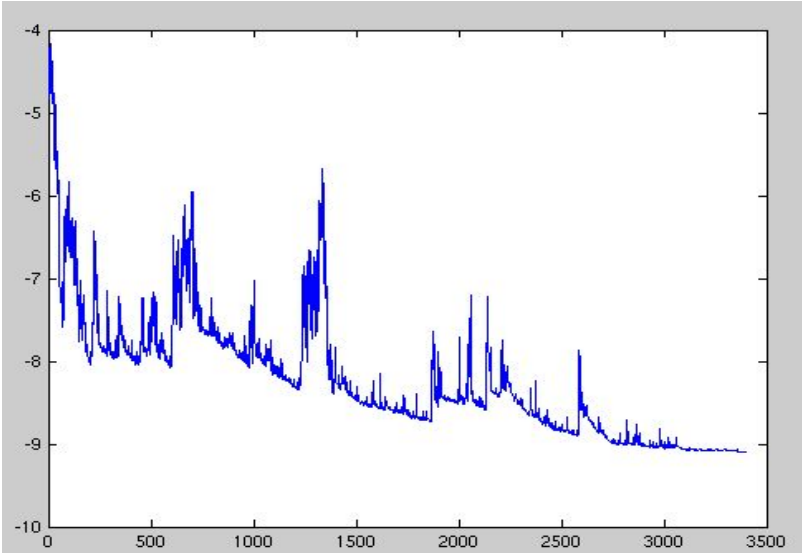
- Based **on how much data we use to compute the gradient** of the **objective function**, there are **three main variants of gradient descent**:
  - **Batch Gradient Descent.**
  - **Stochastic Gradient Descent.**
  - **Mini-Batch Gradient Descent.**

Batch Gradient Descent

- Also known as vanilla gradient descent, computes the gradient of the cost function w.r.t to the parameters $\mathbf{w}$ for the entire training set:
  - Update rule:
    - $w = w - \alpha \nabla_w J(\theta)$.
- It can be very slow and is intractable for datasets that do not fit the memory.

Stochastic Gradient Descent

- In contrast SGD performs a parameter update **for each training example {loss function}**
  i.e. $\boldsymbol{x^i}$ **and label** $\boldsymbol{y^i}$.
- Update Rule:
  - $\boldsymbol{w = w - \alpha \nabla_w J(w; x^i; y^i)}$.
- It is usually much faster compared to BGD, and also can be used to learn online.
- SGD performs frequent updates with a high variance that cause the objective function to fluctuate heavily:
  - It can enable it to jump to new and potential better local minima.
  - It can also ultimately complicate the convergence to the exact minimum.



Mini- Batch Gradient Descent

- It is the mixture of BGD and SGD i.e. it updates the parameter for every mini batch of n training examples:
  - Update Rule:
  - $\mathbf{w = w - \alpha \nabla_w J(w; x^{(i:i+n)}; y^{(i:i+n)}}$.
- Merits:
  - Reduces the variance of the parameter updates, which can lead to more stable convergence;
  - Efficient computation for large models.
- Common mini-batch size range between 50 and 256.

Challenges

- Same learning rate applies to all parameter updates.
  - Getting trapped into suboptimal local minima or saddle points.
  - Choosing a Proper Learning Rate.
    - **Annealing:**
      - Learning rate schedules or reducing the learning rate according to a pre-defined schedule or it becomes smaller than a pre-set threshold.
      - Schedules and threshold values have to be pre-defined, thus may not be able to adapt to a dataset's characteristics.

Thank You!